Static members

Static members are variables, methods, blocks, and nested classes that belong to the **class itself**, rather than to individual objects (instances).

Static Variables (Class Variables)

- Shared by all objects of the class.
- Memory is allocated only once when the class is loaded.
- Accessed using the class name.

```
class Counter {
    static int count = 0;

    static void counter() {
        count++;
        System.out.println(count);
    }
}

public class Main {
    public static void main(String[] args)
{

        Counter.counter();
        Counter.counter();
        Counter.counter();
    }
}

1
2
3
```

```
class Student {
    static String college = "ABC College";
    String name;

    Student(String name) {
        this.name = name;
    }
}

public class Main {
    public static void main(String[] args) {
        Student s1 = new Student("John");
        Student s2 = new Student("Alice");

        System.out.println(Student.college);
        // ABC College
        System.out.println(s1.college);
        System.out.println(s2.college);

    }
}

ABC College
ABC College
ABC College
```

Static Methods

- Belong to the class.
- Can be called without creating an object.
- Can access only static variables and static methods directly.
- Cannot use this or super.

```
class MathUtils {
    static int square(int n) {
        return n * n;
    }
}

public class Main {
    public static void main(String[] args) {
        System.out.println(MathUtils.square(5));
```

```
    }
}

25
```

Can access only static variables and static methods directly

```
class Test {

    static int x = 10;
    int y = 20;

    static void display() {

        System.out.println(x);   // ✔ Allowed

        // System.out.println(y); // ✘ Error
    }
}
```

Static method calling another static method

```
class Test {

    static void show() {
        System.out.println("Show");
    }

    static void display() {
        show();        // ✔ Allowed
    }
}

Show
```

Static Block

- Executes **once** when the class is loaded.
- Used to initialize static variables.
- Executed before the main() method and before any objects are created.
- A static block (also called a **static initialization block**)

class Test {
        static {
            System.out.println("Static block
      executed");
        }
public static void main(String[] args) {
System.out.println("Main method executed");
        }
      }
	class Student {
	    static {
	        System.out.println("Static block");
	    }
	    Student() {
	        System.out.println("Constructor");
	    }
	public static void main(String[] args) {
	        Student s1 = new Student();

Static block executed Main method executed	Student s2 = new Student(); } } Static block Constructor Constructor

class Demo {
    static {
        System.out.println("Static Block 1");
    }
    static {
        System.out.println("Static Block 2");
    }
    public static void main(String[] args) {
        System.out.println("Main Method");
    }
}
Static Block 1
Static Block 2
Main Method
	class Test {
	    static int value;
	
	    static {
	        value = 100;
	        System.out.println("Static block
	executed");
	    }
	
	    public static void main(String[] args) {
	        System.out.println(value);
	    }
	}
	
	Static block executed
	100

Static Nested Class

A class declared as static inside another class.

Can access only the outer class's static members directly

```
class Outer {
    static int x = 10;

    static class Inner {
        void display() {
            System.out.println(x);
        }
    }
}

public class Main {
    public static void main(String[] args) {
        Outer.Inner obj = new Outer.Inner();
        obj.display();
    }
}

10
```

```java
class Outer {
    static int x = 10;

    static class Inner {

        static void display() {
            System.out.println(x);
        }
    }
}

public class Main {
    public static void main(String[] args) {

        // Calling static method of static nested class
        Outer.Inner.display();
    }
}
```

```
10
```

Feature	Static Member	Non-static Member
Belongs to	Class	Object
Memory Allocation	Once	Every object
Access	ClassName.member	object.member
Object Required	No	Yes
Shared	Yes	No